

Django más allá del CRUD

Guía práctica con un proyecto unificado: DuckyArena

0. Objetivo de la sesión

Hasta ahora hemos aprendido los elementos fundamentales de Django:

- Entorno virtual.
- Creación del proyecto.
- Apps.
- URLs.
- Views.
- Models.
- Migraciones.
- Admin.
- Templates.
- Formularios.
- Registro.
- Login.
- Logout.
- Autenticación.

Con todo esto ya podemos crear una aplicación CRUD funcional.

Pero un proyecto real normalmente necesita mucho más:

- Configuración segura.
- Roles y permisos.
- Correos electrónicos.
- Sesiones.
- Gestión de errores.
- Logs.
- Optimización de consultas.
- Archivos.
- Mapas.
- Geolocalización.
- Integraciones externas.
- Inteligencia artificial.
- Machine Learning.
- Tareas que se ejecutan fuera de la petición.
- Aplicaciones móviles.
- Caché.
- Tiempo real.

Django más allá del CRUD

- Tests.
- Herramientas de desarrollo.

Esta sesión pretende responder a una pregunta:

¿Qué cosas interesantes puede hacer Django además de guardar registros en una base de datos y mostrar templates?

1. Proyecto que utilizaremos: DuckyArena

Vamos a imaginar una plataforma llamada:

DuckyArena

En ella tendremos:

Usuarios



Perfiles



Roles

Torneos



Participantes



Localización

Partidas



Resultados



Ranking

Además iremos añadiendo:

Emails

Mapas

Geolocalización

Google Calendar

Inteligencia Artificial

Machine Learning

Django más allá del CRUD

Notificaciones

Sesiones

Logs

Caché

2. Modelo base

Partimos del usuario de Django:

```
from django.contrib.auth.models import User
```

Y añadimos información adicional mediante un perfil.

```
from django.db import models
from django.contrib.auth.models import User
```

```
class ProfileUser(models.Model):
```

```
    ROLE_CHOICES = [
        ("admin", "Administrador"),
        ("organizer", "Organizador"),
        ("player", "Jugador"),
    ]
```

```
    user = models.OneToOneField(
        User,
        on_delete=models.CASCADE,
        related_name="profile"
    )
```

```
    role = models.CharField(
        max_length=20,
        choices=ROLE_CHOICES,
        default="player"
    )
```

```
    bio = models.TextField(
        blank=True
    )
```

```
    avatar = models.ImageField(
        upload_to="avatars/",
        blank=True,
```

Django más allá del CRUD

```
        null=True
    )

    latitude = models.FloatField(
        blank=True,
        null=True
    )

    longitude = models.FloatField(
        blank=True,
        null=True
    )

    def __str__(self):
        return self.user.username
```

¿Por qué utilizamos

OneToOneField

?

Porque queremos esta relación:

User 1 — 1 ProfileUser

Cada usuario tiene como máximo un perfil.

¿Qué significa

related_name="profile"

?

Permite recorrer la relación al revés.

Django más allá del CRUD

Desde `ProfileUser`:

```
profile.user
```

Desde `User`:

```
user.profile
```

Y por tanto:

```
request.user.profile.role
```

Nos devuelve el rol del usuario autenticado.

PARTE I — CONFIGURACIÓN Y BUENAS PRÁCTICAS

3. Variables de entorno

El problema

Muchos alumnos empiezan poniendo esto en `settings.py`:

```
SECRET_KEY = "mi-clave-secreta"
```

Después:

```
EMAIL_HOST_PASSWORD = "mi-password"
```

Y posteriormente:

```
OPENAI_API_KEY = "sk-..."
```

Entonces suben el proyecto a GitHub.

Eso es peligroso.

Las claves privadas nunca deberían formar parte del código fuente.

Django más allá del CRUD

Django más allá del CRUD

4. Crear

.env

Instalamos:

```
pip install python-dotenv
```

Creamos:

```
.env
```

Ejemplo:

```
SECRET_KEY=una-clave-larga  
DEBUG=True
```

```
EMAIL_HOST_USER=correo@gmail.com  
EMAIL_HOST_PASSWORD=password-de-aplicacion
```

```
OPENAI_API_KEY=clave-openai
```

5. Leer

.env

En `settings.py`:

```
import os
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

Ahora:

Django más allá del CRUD

```
SECRET_KEY = os.getenv("SECRET_KEY")
```

Para booleanos:

```
DEBUG = os.getenv(
    "DEBUG",
    "False"
) == "True"
```

¿Qué ocurre paso a paso?

Cuando Django arranca:

```
python manage.py runserver
↓
settings.py
↓
load_dotenv()
↓
lee .env
↓
variables disponibles en os.environ
↓
os.getenv("SECRET_KEY")
```

6.

.gitignore

Nunca subimos `.env`.

```
.env
.venv/
__pycache__/
*.pyc
db.sqlite3
media/
```

Podemos crear:

Django más allá del CRUD

.env.example

con:

```
SECRET_KEY=  
DEBUG=True  
EMAIL_HOST_USER=  
EMAIL_HOST_PASSWORD=  
OPENAI_API_KEY=
```

El alumno que clone el repositorio hace:

.env.example



copiar



.env



rellenar valores

7. Truco: crear settings distintos

En proyectos más avanzados podemos tener:

```
settings/  
├── base.py  
├── development.py  
└── production.py
```

Porque desarrollo y producción no suelen utilizar exactamente la misma configuración.

Por ejemplo:

```
DESARROLLO  
DEBUG=True
```

```
PRODUCCIÓN  
DEBUG=False
```

Django más allá del CRUD

PARTE II — EMAILS Y NOTIFICACIONES

8. Enviar correo desde Django

Django incorpora infraestructura de email.

Para Gmail podemos configurar SMTP.

```
EMAIL_BACKEND = (  
    "django.core.mail.backends.smtp.EmailBackend"  
)
```

```
EMAIL_HOST = "smtp.gmail.com"
```

```
EMAIL_PORT = 587
```

```
EMAIL_USE_TLS = True
```

```
EMAIL_HOST_USER = os.getenv(  
    "EMAIL_HOST_USER"  
)
```

```
EMAIL_HOST_PASSWORD = os.getenv(  
    "EMAIL_HOST_PASSWORD"  
)
```

```
DEFAULT_FROM_EMAIL = EMAIL_HOST_USER
```

Para cuentas de Google es habitual utilizar una contraseña de aplicación en lugar de guardar la contraseña normal de la cuenta.

9. Primer correo

```
from django.core.mail import send_mail
```

```
send_mail(  
    subject="Bienvenido a DuckyArena",  
    message="Tu cuenta se ha creado correctamente.",  
    from_email=None,  
    recipient_list=[  
        "usuario@example.com"  
    ],  
)
```

Django más allá del CRUD

)

Paso a paso

Django recibe:

```
send_mail(...)
```

y hace:

Django

↓

servidor SMTP de Gmail

↓

Gmail

↓

correo del usuario

10. Mandar correo después del registro

```
user = form.save()
```

```
send_mail(  
    "Bienvenido",  
    f"Hola {user.username}",  
    None,  
    [user.email],  
)
```

11. Truco: durante desarrollo no enviar emails reales

Podemos utilizar:

```
EMAIL_BACKEND = (  
    "django.core.mail.backends.console.EmailBackend"  
)
```

Ahora Django imprime el correo en la terminal.

Django más allá del CRUD

Esto es fantástico para clase.

Al registrar un usuario veremos:

Subject: Bienvenido

Hola María,
bienvenida a DuckyArena.

Sin enviar emails reales.

12. Emails HTML

Podemos utilizar:

```
from django.core.mail import EmailMultiAlternatives
```

Pero una estrategia cómoda es generar HTML desde templates.

```
from django.template.loader import render_to_string
```

```
html = render_to_string(  
    "emails/welcome.html",  
    {  
        "user": user  
    }  
)
```

Esto permite reutilizar el conocimiento que ya tienen de templates.

Django más allá del CRUD

PARTE III — PERMISOS Y ROLES

13. Autenticación no es autorización

Es importante diferenciar:

AUTENTICACIÓN

¿Quién eres?

de:

AUTORIZACIÓN

¿Qué puedes hacer?

Un usuario puede estar correctamente autenticado y no tener permiso para determinadas acciones.

14.

login_required

```
from django.contrib.auth.decorators import login_required
```

```
@login_required
def dashboard(request):

    return render(
        request,
        "dashboard.html"
    )
```

Flujo:

GET /dashboard/

↓

@login_required

↓

Django más allá del CRUD

¿autenticado?

↓	↓
sí	no
↓	↓
vista	login

15.

user_passes_test

Ejemplo:

```
from django.contrib.auth.decorators import user_passes_test
```

```
def is_staff(user):
```

```
    return user.is_staff
```

```
@user_passes_test(is_staff)
```

```
def staff_area(request):
```

```
    ...
```

16. Nuestro decorador

role_required

Creamos:

```
accounts/decorators.py
```

```
from functools import wraps
```

```
from django.contrib import messages
```

Django más allá del CRUD

```
from django.shortcuts import redirect
```

```
def role_required(*roles):

    def decorator(view_func):

        @wraps(view_func)
        def wrapper(
            request,
            *args,
            **kwargs
        ):

            if not request.user.is_authenticated:

                return redirect("login")

            if (
                request.user.profile.role
                not in roles
            ):

                messages.error(
                    request,
                    "No tienes permisos."
                )

                return redirect("home")

            return view_func(
                request,
                *args,
                **kwargs
            )

        return wrapper

    return decorator
```

Uso:

```
@role_required(
    "admin",
    "organizer"
)
```

Django más allá del CRUD

def create_tournament(request):

...

Paso a paso

Supongamos:

request.user.username

es:

maria

Django consulta:

request.user.profile

Después:

request.user.profile.role

Resultado:

player

El decorador está permitiendo:

admin

organizer

Como:

player ∈ [admin, organizer]

redireccionamos.

17. Groups y Permissions

En proyectos reales no siempre necesitamos inventar nuestro propio sistema de roles.

Django tiene:

Users

Django más allá del CRUD

Groups

Permissions

Ejemplo:

```
request.user.has_perm(  
    "arena.add_tournament"  
)
```

También:

```
request.user.groups.filter(  
    name="Organizadores"  
)exists()
```

Django 5.2 sigue proporcionando su sistema integrado de permisos por usuario, grupo y modelo. ([Django Project](#))

18. Truco importante de seguridad

Esto:

```
{% if user.profile.role == "admin" %}
```

```
<button>  
    Eliminar usuario  
</button>
```

```
{% endif %}
```

NO protege nada.

Solo oculta el botón.

El usuario podría acceder directamente:

```
/accounts/delete/12/
```

La comprobación real debe estar en el servidor:

```
@role_required("admin")  
def delete_user(request, id):
```

```
...
```

Django más allá del CRUD

PARTE IV — MESSAGES

19. Messages Framework

Sirve para mostrar mensajes que viven durante una petición/redirección.

Ejemplos:

Usuario creado.
Datos actualizados.
No tienes permisos.
Torneo eliminado.

Vista:

```
from django.contrib import messages
```

```
messages.success(  
    request,  
    "Torneo creado."  
)
```

```
return redirect(  
    "tournament_list"  
)
```

20. Mostrarlo en

base.html

```
{% if messages %}
```

```
    {% for message in messages %}
```

```
        <div class="alert alert-{{ message.tags }}">  
            {{ message }}  
        </div>
```

Django más allá del CRUD

```
{% endfor %}
```

```
{% endif %}
```

Como está en `base.html`, funciona en toda la aplicación.

21. Tip: niveles

Tenemos:

```
messages.debug()  
messages.info()  
messages.success()  
messages.warning()  
messages.error()
```

Podemos hacer que visualmente sean distintos utilizando Bootstrap.

PARTE V — SESIONES

22. ¿Qué es una sesión?

HTTP no recuerda automáticamente las peticiones anteriores.

```
GET /home/  
GET /tournament/1/  
GET /profile/
```

Son peticiones independientes.

La sesión permite mantener información asociada al visitante.

23. Guardar datos

```
request.session[
```

Django más allá del CRUD

```
"theme"  
] = "dark"
```

Leer:

```
theme = request.session.get(  
    "theme",  
    "light"  
)
```

Eliminar:

```
del request.session["theme"]
```

24. Último torneo visitado

En la vista:

```
def tournament_detail(  
    request,  
    tournament_id  
):  
  
    tournament = get_object_or_404(  
        Tournament,  
        id=tournament_id  
    )  
  
    request.session[  
        "last_tournament"  
    ] = tournament.id  
  
    ...
```

Después en dashboard:

```
last_id = request.session.get(  
    "last_tournament"  
)
```

25. Mini carrito

Django más allá del CRUD

Las sesiones permiten enseñar un carrito sin crear tablas nuevas.

```
cart = request.session.get(
    "cart",
    []
)

cart.append(product.id)

request.session["cart"] = cart
```

PARTE VI — MIDDLEWARE

26. ¿Qué es middleware?

Un middleware está en medio del navegador y nuestras views.

```
REQUEST
↓
Middleware A
↓
Middleware B
↓
View
↓
Middleware B
↓
Middleware A
↓
RESPONSE
```

Esto permite ejecutar lógica para muchas páginas sin repetir código.

27. Middleware de tiempo de respuesta

```
import time
```

```
class ResponseTimeMiddleware:
```

Django más allá del CRUD

```
def __init__(
    self,
    get_response
):

    self.get_response = (
        get_response
    )

def __call__(
    self,
    request
):

    start = time.time()

    response = (
        self.get_response(
            request
        )
    )

    elapsed = (
        time.time() - start
    )

    print(
        request.path,
        f'{elapsed:.4f}s'
    )

    return response
```

28. Registrar middleware

`settings.py`:

```
MIDDLEWARE = [
    ...
    "config.middleware.ResponseTimeMiddleware",
]
```

Ahora:

Django más allá del CRUD

GET /arena/ 0.0127s

GET /profile/ 0.0083s

POST /login/ 0.0212s

PARTE VII — SIGNALS

29. ¿Qué problema solucionan?

Queremos que:

crear User

↓

crear ProfileUser

sin acordarnos de hacerlo manualmente en todas partes.

30. Signal

post_save

```
from django.db.models.signals import (
    post_save
)
```

```
from django.dispatch import receiver
```

```
from django.contrib.auth.models import (
    User
)
```

```
from .models import ProfileUser
```

```
@receiver(
    post_save,
    sender=User
```

Django más allá del CRUD

```
)
def create_profile(
    sender,
    instance,
    created,
    **kwargs
):

    if created:

        ProfileUser.objects.create(
            user=instance
        )
```

Paso a paso

Cuando hacemos:

```
User.objects.create_user(
    username="ana",
    password="12345678"
)
```

ocurre:

```
INSERT User
  ↓
post_save
  ↓
created=True
  ↓
create_profile()
  ↓
INSERT ProfileUser
```

PARTE VIII — CONTEXT PROCESSORS

31. Datos globales en templates

Imaginemos que en todas las páginas queremos mostrar:

Django más allá del CRUD

Usuarios registrados: 528

No queremos hacer en todas las views:

```
{
    "total_users":
        User.objects.count()
}
```

Creamos:

```
def global_stats(request):

    return {
        "total_users":
            User.objects.count()
    }
```

Y lo registramos como context processor.

Después cualquier template puede usar:

```
{{ total_users }}
```

PARTE IX — TEMPLATE TAGS Y FILTERS

32. Crear nuestros propios filtros

```
from django import template
```

```
register = template.Library()
```

```
@register.filter
```

```
def role_name(value):
```

```
    roles = {
        "admin": "Administrador",
        "organizer": "Organizador",
        "player": "Jugador",
```

Django más allá del CRUD

```
}
```

```
return roles.get(  
    value,  
    value  
)
```

Template:

```
{% load arena_filters %}
```

```
{{ user.profile.role|role_name }}
```

PARTE X — ERRORES

33.

get_object_or_404

En vez de:

```
Tournament.objects.get(  
    id=id  
)
```

usamos:

```
get_object_or_404(  
    Tournament,  
    id=id  
)
```

Si no existe:

HTTP 404

34. Errores que deben conocer

Django más allá del CRUD

200 → OK
201 → creado
302 → redirección
400 → petición incorrecta
401 → autenticación requerida
403 → prohibido
404 → no encontrado
500 → error servidor

Estos códigos serán especialmente importantes cuando entremos en APIs.

PARTE XI — ORM AVANZADO

35.

Q

Queremos buscar por nombre O descripción.

```
from django.db.models import Q
```

```
Tournament.objects.filter(  
    Q(name__icontains=query)  
    |  
    Q(description__icontains=query)  
)
```

36. Lookups

```
score__gt=100
```

Mayor que 100.

```
score__gte=100
```

Django más allá del CRUD

Mayor o igual.

```
name__icontains="django"
```

Contiene texto ignorando mayúsculas/minúsculas.

```
created_at__year=2026
```

Fecha del año indicado.

37.

aggregate

Queremos la media:

```
from django.db.models import Avg
```

```
result = Game.objects.aggregate(  
    average=Avg("score")  
)
```

38.

annotate

Número de jugadores por torneo:

```
from django.db.models import Count
```

```
tournaments = (  
    Tournament.objects  
    .annotate(  
        num_players=Count('players')  
    )  
)
```

Django más allá del CRUD

```
        player_count=Count(
            "players"
        )
    )
)
```

Ahora cada tournament tiene temporalmente:

```
tournament.player_count
```

39.

F

Actualizar directamente en SQL:

```
from django.db.models import F
```

```
PlayerStats.objects.update(
    score=F("score") + 10
)
```

No necesitamos leer cada objeto primero.

PARTE XII — OPTIMIZACIÓN

40. Problema N+1

Tenemos 100 torneos.

Cada uno tiene:

```
tournament.owner
```

Si hacemos:

Django más allá del CRUD

```
tournaments = (  
    Tournament.objects.all()  
)
```

for tournament in tournaments:

```
    print(  
        tournament.owner.username  
    )
```

podríamos provocar muchas consultas.

41.

`select_related`

```
Tournament.objects.select_related(  
    "owner"  
)
```

Es apropiado principalmente para relaciones:

ForeignKey
OneToOneField

42.

`prefetch_related`

Para:

ManyToMany
relaciones inversas

podemos utilizar:

Django más allá del CRUD

```
Tournament.objects.prefetch_related(  
    "players"  
)
```

Django documenta estas herramientas dentro de sus mecanismos oficiales de optimización de acceso a base de datos. ([Django Project](#))

PARTE XIII — PAGINACIÓN

43. No mostrar 10.000 registros

```
from django.core.paginator import (  
    Paginator  
)
```

```
tournaments = (  
    Tournament.objects.all()  
)
```

```
paginator = Paginator(  
    tournaments,  
    10  
)
```

```
page = request.GET.get(  
    "page"  
)
```

```
page_obj = (  
    paginator.get_page(page)  
)
```

Flujo

```
/tournaments/?page=3  
    ↓  
request.GET["page"]  
    ↓  
3
```

Django más allá del CRUD

↓
Paginator
↓
registros 21-30

PARTE XIV — LOGGING

44. Dejar de utilizar

print

En aprendizaje hacemos:

```
print("Usuario creado")
```

Mejor:

```
import logging
```

```
logger = logging.getLogger(  
    __name__  
)
```

```
logger.info(  
    "Usuario creado"  
)
```

45. Niveles

```
DEBUG  
INFO  
WARNING  
ERROR  
CRITICAL
```

Django más allá del CRUD

Ejemplo:

```
logger.warning(  
    "Intento de acceso no autorizado"  
)
```

PARTE XV — ARCHIVOS E IMÁGENES

46.

ImageField

```
avatar = models.ImageField(  
    upload_to="avatars/",  
    blank=True,  
    null=True  
)
```

Instalamos:

```
pip install pillow
```

47. MEDIA

```
MEDIA_URL = "/media/"
```

```
MEDIA_ROOT = BASE_DIR / "media"
```

Formulario:

```
<form  
    method="POST"  
    enctype="multipart/form-data"  
>
```

Vista:

```
form = ProfileForm(
```

Django más allá del CRUD

```
request.POST,  
request.FILES  
)
```

El detalle de:

`request.FILES`

es importante.

`request.POST` contiene campos normales.

`request.FILES` contiene archivos.

PARTE XVI — MAPAS

48. Introducir mapas en Django

Este puede ser uno de los ejemplos más visuales de la sesión.

Podemos mostrar los torneos sobre un mapa.

Por ejemplo:

DuckyArena

📍 Madrid
Tournament A

📍 Valencia
Tournament B

📍 Sevilla
Tournament C

Una opción muy cómoda para enseñar es **Leaflet.js**.

Django continúa gestionando:

datos
coordenadas

Django más allá del CRUD

usuarios

torneos

y JavaScript dibuja el mapa.

49. Añadir coordenadas al torneo

```
class Tournament(models.Model):
```

```
    name = models.CharField(
        max_length=150
    )
```

```
    latitude = models.FloatField(
        null=True,
        blank=True
    )
```

```
    longitude = models.FloatField(
        null=True,
        blank=True
    )
```

Ejemplo:

Madrid

latitude:
40.4168

longitude:
-3.7038

50. Mapa básico con Leaflet

En el template podemos cargar Leaflet y crear:

```
<div
    id="map"
    style="height: 500px;"
></div>
```

Django más allá del CRUD

JavaScript:

```
const map = L.map("map")
    .setView(
        [40.4168, -3.7038],
        6
    );

L.tileLayer(
    "https://tile.openstreetmap.org/{z}/{x}/{y}.png"
).addTo(map);
```

Añadimos marcador:

```
L.marker(
    [40.4168, -3.7038]
)
.addTo(map)
.bindPopup(
    "Ducky Tournament Madrid"
);
```

51. Pasar coordenadas de Django a JavaScript

Vista:

```
tournaments = (
    Tournament.objects
        .exclude(latitude=None)
        .exclude(longitude=None)
)
```

Template:

```
{% for tournament in tournaments %}
```

```
L.marker([
    {{ tournament.latitude }},
    {{ tournament.longitude }}
])
.addTo(map)
.bindPopup(
```

Django más allá del CRUD

```
"{{ tournament.name }}"  
);
```

```
{% endfor %}
```

Conceptualmente:

```
DATABASE  
↓  
Django Model  
↓  
View  
↓  
Template  
↓  
JavaScript  
↓  
Leaflet  
↓  
Mapa
```

PARTE XVII — GEOLOCALIZACIÓN DEL USUARIO

52. Obtener ubicación desde el navegador

Los navegadores modernos ofrecen:

```
navigator.geolocation
```

Ejemplo:

```
navigator.geolocation  
.getCurrentPosition(  
  function(position) {  
  
    console.log(  
      position.coords.latitude  
    );
```

Django más allá del CRUD

```
        console.log(  
            position.coords.longitude  
        );  
    }  
);
```

El navegador pedirá permiso al usuario.

53. Mostrar al usuario en el mapa

```
navigator.geolocation  
.getCurrentPosition(  
    function(position) {  
  
        const lat =  
            position.coords.latitude;  
  
        const lng =  
            position.coords.longitude;  
  
        L.marker([  
            lat,  
            lng  
        ])  
        .addTo(map)  
        .bindPopup(  
            "Estás aquí"  
        )  
        );  
  
        map.setView(  
            [lat, lng],  
            13  
        );  
    }  
);
```

Esto produce una demo muy atractiva en clase.

54. Guardar ubicación en Django

Django más allá del CRUD

Podemos enviar:

latitude
longitude

a una view.

Modelo:

profile.latitude = latitude
profile.longitude = longitude

profile.save()

Así podríamos crear:

Torneos cerca de mí

55. GeoDjango

Si queremos llevar mapas mucho más lejos, Django dispone de **GeoDjango**.

Permite trabajar con:

puntos
distancias
polígonos
zonas
intersecciones

Con bases de datos geográficas como PostGIS.

Un ejemplo conceptual sería:

Mostrar torneos
a menos de 20 km
del usuario

Para una sesión inicial dejaría GeoDjango como demostración o ampliación y utilizaría latitud/longitud + Leaflet para la parte práctica.

Django más allá del CRUD

PARTE XVIII — CONVERTIR DJANGO EN UNA APP “MÓVIL”

56. Primera opción: diseño responsive

Antes de crear una app Android/iOS, una aplicación Django puede ser perfectamente usable desde móvil.

Con Bootstrap:

```
<meta
  name="viewport"
  content="width=device-width,
    initial-scale=1"
>
```

Y componentes responsive.

57. Segunda opción: PWA

Podemos transformar la web en una:

Progressive Web App

Una PWA puede:

- instalarse en el móvil
- tener icono
- abrirse a pantalla completa
- utilizar caché
- funcionar parcialmente offline
- recibir notificaciones en determinados escenarios

Necesitamos principalmente:

- manifest.json
- service worker
- HTTPS en producción

Django más allá del CRUD

58.

manifest.json

Ejemplo:

```
{  
  "name": "DuckyArena",  
  "short_name": "Ducky",  
  "start_url": "/",  
  "display": "standalone"  
}
```

Añadimos iconos y colores según necesidad.

Ahora el navegador puede ofrecer:

Añadir a pantalla de inicio

Esto es una manera fantástica de explicar:

No toda aplicación móvil tiene que empezar creando una aplicación nativa.

59. App móvil real

Más adelante, cuando aprendan DRF, podremos hacer:

```
Django  
↓  
DRF API  
↓  
JSON  
↓  
Flutter / React Native  
↓  
Android + iOS
```

Por eso conviene guardar esta parte profunda para después de DRF.

Hoy podemos enseñarles la arquitectura.

Django más allá del CRUD

PARTE XIX — MACHINE LEARNING

60. Integrar un modelo de Machine Learning

Django puede utilizar prácticamente cualquier librería Python.

Por ejemplo:

scikit-learn
pandas
numpy
PyTorch
TensorFlow

Supongamos que queremos predecir:

¿Cuántos jugadores asistirán a un torneo?

Podemos tener un modelo de ML entrenado con datos históricos.

61. Ejemplo muy sencillo con scikit-learn

Datos históricos:

precio plazas inscritos

0	16	14
10	16	11
0	32	28
20	32	18

Entrenamos fuera de Django:

```
from sklearn.linear_model import (  
    LinearRegression  
)
```

Django más allá del CRUD

```
import joblib
```

```
X = [  
    [0, 16],  
    [10, 16],  
    [0, 32],  
    [20, 32],  
]
```

```
y = [  
    14,  
    11,  
    28,  
    18,  
]
```

```
model = LinearRegression()
```

```
model.fit(  
    X,  
    y  
)
```

```
joblib.dump(  
    model,  
    "attendance_model.joblib"  
)
```

62. Cargar el modelo desde Django

```
import joblib
```

```
model = joblib.load(  
    "attendance_model.joblib"  
)
```

Predicción:

```
prediction = model.predict(  
    [[10, 32]]
```

Django más allá del CRUD

)

Resultado aproximado:

23 participantes

63. View con ML

```
def tournament_prediction(
    request,
    tournament_id
):

    tournament = get_object_or_404(
        Tournament,
        id=tournament_id
    )

    prediction = model.predict([
        [
            tournament.price,
            tournament.max_players
        ]
    ])[0]

    return render(
        request,
        "prediction.html",
        {
            "prediction":
                round(prediction)
        }
    )
```

Arquitectura

Usuario



Django View



datos del Tournament



modelo Machine Learning

Django más allá del CRUD

↓
predict()
↓
resultado
↓
Template

64. Ideas de ML para un proyecto

En DuckyArena podríamos:

predecir asistencia
predecir abandono
clasificar jugadores
detectar comportamiento anómalo
recomendar torneos
predecir puntuaciones
crear matchmaking

Esto permite que los alumnos entiendan una idea importante:

Django no “hace Machine Learning”. Django puede convertirse en la aplicación web que utiliza un modelo de Machine Learning.

PARTE XX — GOOGLE CALENDAR

65. Crear eventos automáticamente

Supongamos que un usuario se apunta a:

Ducky Championship
15 septiembre
18:00

Podemos ofrecer:

Añadir a Google Calendar

Django más allá del CRUD

La API de Google Calendar permite gestionar eventos, calendarios, horarios y asistentes desde aplicaciones externas. ([Google Developers](#))

66. Preparación

Actualmente el quickstart oficial de Google para Python utiliza las bibliotecas:

```
pip install \
google-api-python-client \
google-auth-httpplib2 \
google-auth-oauthlib
```

y requiere habilitar Calendar API en un proyecto de Google Cloud y configurar OAuth. ([Google Developers](#))

67. OAuth

Este es un concepto interesante para introducir:

```
Usuario
↓
DuckyArena
↓
"¿Permites acceder a tu Calendar?"
↓
Google
↓
Usuario acepta
↓
DuckyArena recibe autorización
```

No estamos pidiendo al usuario:

contraseña de Google

Estamos utilizando OAuth.

Django más allá del CRUD

68. Crear un evento

Una vez tenemos el cliente autorizado, podemos construir:

```
event = {  
  
    "summary":  
        "Ducky Championship",  
  
    "description":  
        "Torneo de DuckyArena",  
  
    "start": {  
        "dateTime":  
            "2026-09-15T18:00:00+02:00",  
        "timeZone":  
            "Europe/Madrid",  
    },  
  
    "end": {  
        "dateTime":  
            "2026-09-15T20:00:00+02:00",  
        "timeZone":  
            "Europe/Madrid",  
    },  
}  
  
Y:  
  
event = (  
    service  
    .events()  
    .insert(  
        calendarId="primary",  
        body=event  
    )  
    .execute()  
)
```

La API oficial utiliza `events().insert()` para crear eventos. ([Google Developers](#))

Django más allá del CRUD

69. Idea de ejercicio

Cuando el alumno pulsa:

Inscribirme

hacemos:

guardar inscripción

↓

messages.success

↓

enviar email

↓

opcionalmente

crear evento Calendar

Aquí tienen una integración completa con un servicio externo.

PARTE XXI — OPENAI

70. Integrar un modelo de OpenAI

Django puede actuar como backend para una aplicación con IA.

Ejemplos para DuckyArena:

generar descripción de torneo

resumir reglas

moderación asistida

FAQ inteligente

crear emails

clasificar comentarios

explicar estadísticas

crear recomendaciones

La API actual de OpenAI utiliza la **Responses API** para generar respuestas a partir de entradas de texto, imágenes o archivos, además de poder trabajar con herramientas.

([OpenAI Developers](#))

Django más allá del CRUD

71. Instalar SDK

`pip install openai`

En `.env`:

`OPENAI_API_KEY=...`

No ponemos nunca la clave directamente en JavaScript o en el navegador.

72. Crear cliente

`from openai import OpenAI`

`client = OpenAI()`

El SDK puede obtener la clave desde:

`OPENAI_API_KEY`

del entorno.

73. Generar descripción de torneo

Por ejemplo:

```
response = client.responses.create(
    model="gpt-5.6",
    input=(
        "Genera una descripción breve "
        "y atractiva para un torneo "
        "de programación Django."
    )
)
```

Y obtenemos el texto:

```
description = (
    response.output_text
```

Django más allá del CRUD

)

Los modelos actuales de OpenAI están disponibles mediante la Responses API y SDKs oficiales. ([Plataforma OpenAI](#))

74. View Django

```
@login_required
def generate_description(
    request
):

    topic = request.POST.get(
        "topic"
    )

    response = (
        client.responses.create(
            model="gpt-5.6",
            input=(
                "Genera una descripción "
                "de torneo sobre: "
                + topic
            )
        )
    )

    text = response.output_text

    return render(
        request,
        "generated.html",
        {
            "text": text
        }
    )
```

75. Flujo

Usuario

↓

Django más allá del CRUD

formulario
↓
POST
↓
Django View
↓
OpenAI API
↓
respuesta
↓
Django
↓
Template

Es importante que la petición a OpenAI salga desde el servidor.

Nunca:

HTML/JavaScript
↓
API key secreta
↓
OpenAI

porque estaríamos exponiendo la clave.

76. Ejemplo más interesante

Tenemos comentarios:

"El torneo estuvo genial,
pero tuvimos que esperar
demasiado para empezar."

Podemos pedir:

Clasifica este comentario:

positivo
negativo
mixto

Resultado:

Django más allá del CRUD

mixto

Podemos guardar el resultado en Django.

77. Diferencia ML tradicional vs modelo generativo

Es una excelente pregunta para clase.

Machine Learning propio

nuestros datos

↓

entrenamiento

↓

modelo

↓

predict

Modelo externo de IA

prompt

↓

API

↓

modelo ya entrenado

↓

respuesta

PARTE XXII — CONSUMIR APIs EXTERNAS

78. Django como cliente HTTP

Podemos consumir prácticamente cualquier API.

Django más allá del CRUD

Una librería clásica es:

```
pip install requests
```

Por ejemplo:

```
import requests
```

```
response = requests.get(
    "https://api.example.com/data"
)
```

```
data = response.json()
```

79. Otra opción: HTTPX

```
pip install httpx
```

Uso:

```
import httpx
```

```
response = httpx.get(
    "https://api.example.com/data"
)
```

HTTPX resulta interesante cuando más adelante trabajemos con código async.

PARTE XXIII — CACHÉ

80. ¿Qué problema resuelve?

Supongamos que tenemos:

Ranking global

y calcularlo requiere:

Django más allá del CRUD

50 consultas SQL

pero solo cambia cada pocos minutos.

No queremos recalcularlo en cada petición.

81. Caché simple

Podemos utilizar:

```
from django.core.cache import cache
ranking = cache.get(
    "global_ranking"
)
```

Si no existe:

if ranking is None:

```
    ranking = (
        calculate_ranking()
    )

    cache.set(
        "global_ranking",
        ranking,
        300
    )
```

Flujo

Primera petición:

```
cache vacío
↓
calcular
↓
guardar 5 min
```

Siguiente petición:

cache

Django más allá del CRUD

↓
resultado inmediato

82. Redis

Para proyectos reales es muy frecuente utilizar:

Redis

para:

caché

sesiones

colas

Celery

datos temporales

Es una buena tecnología para mencionar aunque no la instalemos durante la sesión.

PARTE XXIV — TAREAS EN SEGUNDO PLANO

83. El problema

Imaginemos esta view:

crear torneo

↓

enviar 500 emails

↓

generar PDF

↓

consultar IA

↓

response

El navegador podría esperar muchísimo.

Django más allá del CRUD

Las operaciones lentas no deberían bloquear necesariamente una petición HTTP.

84. Celery

Una herramienta habitual en Django es:

Celery

Arquitectura:

Django
↓
cola
↓
Celery worker
↓
ejecuta tarea

Ejemplo conceptual:

```
send_tournament_emails.delay(  
    tournament.id  
)
```

Django responde rápidamente.

Otro proceso envía los emails.

85. Casos para tareas en background

enviar correos
generar informes
procesar imágenes
procesar vídeos
ejecutar ML pesado
consultar servicios lentos
crear PDFs
generar estadísticas

Django más allá del CRUD

PARTE XXV — WEBSOCKETS Y TIEMPO REAL

86. HTTP normal

Normalmente:

cliente pregunta



servidor responde

Pero imaginemos un chat.

Queremos:

Jugador A escribe



Jugador B lo recibe inmediatamente

87. Django Channels

Una herramienta del ecosistema Django permite trabajar con:

WebSockets

Casos:

chat

notificaciones en tiempo real

marcadores deportivos

estado de partidas

usuarios conectados

dashboards vivos

Conceptualmente:

Browser



WebSocket



Django Channels

Django más allá del CRUD

Esto puede ser una demo muy atractiva para cursos más avanzados.

PARTE XXVI — LIBRERÍAS ÚTILES

88. django-extensions

`pip install django-extensions`

Nos proporciona herramientas adicionales.

Muy útil:

`python manage.py shell_plus`

En lugar de importar modelos manualmente.

89. django-debug-toolbar

Una de las herramientas que más merece la pena enseñar.

Permite ver información de cada página:

tiempo
SQL
templates
headers
cache
signals

Especialmente útil para detectar:

N+1 queries

90. Pillow

`pip install pillow`

Django más allá del CRUD

Para:

ImageField

91. Requests

`pip install requests`

Para consumir APIs externas.

92. HTTPX

`pip install httpx`

Ciente HTTP moderno con soporte sync/async.

93. django-crispy-forms

Puede ayudar a renderizar formularios de manera más cómoda.

Especialmente cuando se combina con Bootstrap.

94. django-allauth

Interesante para autenticación avanzada.

Permite construir escenarios como:

Login con Google

Login con GitHub

registro

verificación de email

gestión de cuentas

Esto podría ser otra clase completa.

Django más allá del CRUD

95. WhiteNoise

Útil para servir archivos estáticos en determinados despliegues sencillos.

Ejemplo:

CSS
JavaScript
imágenes estáticas

96. Celery

Para tareas asíncronas.

97. Redis

Para:

cache
colas
sesiones
Celery
Channels

98. pandas

pip install pandas

Django puede utilizarlo para:

informes
Excel
estadísticas
limpieza de datos
Machine Learning

Django más allá del CRUD

99. openpyxl

Muy útil para:

leer Excel
crear Excel
exportar informes

Ejemplo:

DuckyArena
↓
exportar ranking
↓
ranking.xlsx

100. ReportLab / WeasyPrint

Para generar PDFs.

Ejemplo:

Tournament
↓
Generar certificado
↓
PDF

PARTE XXVII — EXPORTAR DATOS

101. CSV

Podemos crear una descarga:

```
import csv
```

```
from django.http import (
```

Django más allá del CRUD

```
        HttpResponseRedirect
    )

def export_users(request):

    response = HttpResponseRedirect(
        content_type="text/csv"
    )

    writer = csv.writer(
        response
    )

    writer.writerow([
        "Username",
        "Email",
    ])

    for user in User.objects.all():

        writer.writerow([
            user.username,
            user.email,
        ])

    return response
```

102. JSON sin DRF

Antes de enseñar DRF podemos demostrar que Django ya puede devolver JSON.

```
from django.http import JsonResponse
```

```
def stats(request):

    return JsonResponse({
        "users":
            User.objects.count(),

        "tournaments":
            Tournament.objects.count(),
    })
```

Django más allá del CRUD

Accedemos:

/stats/

Resultado:

```
{  
  "users": 120,  
  "tournaments": 35  
}
```

Y aquí podemos parar.

Porque la pregunta lógica de un alumno será:

Entonces, ¿podemos construir toda una API?

Respuesta:

Sí. Y para hacerlo bien utilizaremos Django REST Framework.

Perfecto puente para la semana siguiente.

PARTE XXVIII — AJAX / FETCH

103. Actualizar información sin recargar

JavaScript puede llamar a Django.

```
fetch("/stats/")  
  .then(  
    response =>  
      response.json()  
  )  
  .then(  
    data =>  
      console.log(data)  
  );
```

Ahora:

JavaScript



Django más allá del CRUD

Django

↓

JSON

↓

JavaScript

Sin recargar toda la página.

104. Contador dinámico

HTML:

```
<span id="users">  
</span>
```

JavaScript:

```
fetch("/stats/")  
.then(  
  response => response.json()  
)  
.then(  
  data => {  
  
    document  
      .getElementById("users")  
      .textContent =  
        data.users;  
  
  }  
);
```

Es un anticipo fantástico de frontend + API.

PARTE XXIX — MANAGEMENT COMMANDS

Django más allá del CRUD

105. Crear comandos propios

Podemos crear:

```
python manage.py seed
```

Para rellenar datos.

Estructura:

```
arena/  
└── management/  
    ├── commands/  
    └── seed.py
```

106. Ejemplo

```
from django.core.management.base import (  
    BaseCommand  
)
```

```
from arena.models import Tournament
```

```
class Command(BaseCommand):
```

```
    def handle(  
        self,  
        *args,  
        **options  
    ):
```

```
        for i in range(20):
```

```
            Tournament.objects.create(  
                name=f"Torneo {i}"  
            )
```

```
        self.stdout.write(  
            self.style.SUCCESS(  
                "Datos creados"  
            )  
        )
```

Django más allá del CRUD

PARTE XXX — FIXTURES

107. Exportar

```
python manage.py dumpdata arena \
--indent 2 \
> arena.json
```

Importar:

```
python manage.py loaddata arena.json
```

Muy útil para preparar ejercicios.

PARTE XXXI — TESTING

108. Primer test

```
from django.test import TestCase
```

```
class DashboardTest(TestCase):
```

```
    def test_redirects_without_login(
        self
    ):
```

```
        response = self.client.get(
            "/accounts/dashboard/"
        )
```

```
        self.assertEqual(
            response.status_code,
            302
        )
```

109. Test con usuario

Django más allá del CRUD

```
class DashboardTest(TestCase):
```

```
    def setUp(self):
```

```
        self.user = (
            User.objects.create_user(
                username="ana",
                password="12345678"
            )
        )
```

```
    def test_dashboard(
        self
    ):
```

```
        self.client.login(
            username="ana",
            password="12345678"
        )
```

```
        response = (
            self.client.get(
                "/accounts/dashboard/"
            )
        )
```

```
        self.assertEqual(
            response.status_code,
            200
        )
```

PARTE XXXII — TRUCOS RÁPIDOS DE ORM

110.

.exists()

Django más allá del CRUD

En vez de:

```
if queryset.count() > 0:
```

usar:

```
if queryset.exists():
```

cuando solo queremos saber si existe algún resultado.

111.

`.first()`

```
user = (  
    User.objects  
    .filter(  
        username="ana"  
    )  
    .first()  
)
```

Si no existe:

None

No genera `DoesNotExist`.

112.

`get_or_create`

```
profile, created = (  
    ProfileUser.objects  
    .get_or_create(  

```

Django más allá del CRUD

```
        user=request.user
    )
)
```

113.

update_or_create

PlayerStats.objects.update_or_create(

```
    user=request.user,

    defaults={
        "score": 500
    }
)
```

114.

bulk_create

En vez de:

for i in range(1000):

```
    Tournament.objects.create(
        name=f"Torneo {i}"
    )
```

podemos construir:

```
objects = [
    Tournament(
        name=f"Torneo {i}"
    )
```

Django más allá del CRUD

```
)  
    for i in range(1000)  
]
```

y:

```
Tournament.objects.bulk_create(  
    objects  
)
```

Muy útil para explicar rendimiento.

115.

only

Si tenemos un modelo enorme:

```
Tournament.objects.only(  
    "name"  
)
```

podemos limitar inicialmente campos cargados.

116.

values

```
Tournament.objects.values(  
    "id",  
    "name"  
)
```

Resultado:

Django más allá del CRUD

```
[
  {
    "id": 1,
    "name": "Ducky Cup"
  }
]
```

Esto empieza a parecerse bastante a los datos que devolverá una API.

117.

values_list

```
User.objects.values_list(
    "username",
    flat=True
)
```

Resultado:

```
[
    "ana",
    "maria",
    "pedro"
]
```

PARTE XXXIII — TRUCOS DE TEMPLATES

118.

default

Django más allá del CRUD

```
{{ user.first_name|default:"Sin nombre" }}
```

119.

truncatechars

```
{{ tournament.description|truncatechars:100 }}
```

120.

date

```
{{ tournament.created_at|date:"d/m/Y H:i" }}
```

121.

pluralize

```
{{ players.count }}  
jugador{{ players.count|pluralize:"es" }}
```

122.

with

Django más allá del CRUD

Evita repetir expresiones largas:

```
{% with profile=user.profile %}
```

```
    {{ profile.role }}
```

```
{% endwith %}
```

PARTE XXXIV — TRUCOS DE URLs

123. Nunca hardcodear URLs

Mal:

```
<a href="/accounts/profile/">
```

Mejor:

```
<a href="{% url 'accounts:profile' %}">
```

124. Namespaces

`accounts/urls.py`:

```
app_name = "accounts"
```

Después:

```
{% url "accounts:dashboard" %}
```

Esto evita colisiones entre apps.

PARTE XXXV — TRUCOS DE ADMIN

125. Mejorar Django Admin

Django más allá del CRUD

No tenemos por qué dejar:

```
admin.site.register(
    Tournament
)
```

Podemos hacer:

```
@admin.register(Tournament)
class TournamentAdmin(
    admin.ModelAdmin
):
```

```
    list_display = [
        "name",
        "owner",
        "created_at",
    ]
```

```
    search_fields = [
        "name",
        "owner__username",
    ]
```

```
    list_filter = [
        "created_at",
    ]
```

Ahora tenemos búsqueda y filtros automáticamente.

126.

readonly_fields

```
readonly_fields = [
    "created_at"
]
```

Django más allá del CRUD

127.

list_per_page

list_per_page = 50

PARTE XXXVI — SEGURIDAD

128. CSRF

Todo formulario POST:

```
<form method="POST">
```

```
    {% csrf_token %}
```

```
</form>
```

129. SQL Injection

Una ventaja del ORM:

```
User.objects.filter(
    username=username
)
```

Django parametriza la consulta.

No deberíamos construir SQL concatenando entradas del usuario.

130. XSS

Django escapa automáticamente contenido en templates.

Django más allá del CRUD

Si el usuario escribe:

```
<script>
alert("hola")
</script>
```

Django normalmente no debería interpretarlo como HTML ejecutable al mostrarlo mediante `{{ variable }}`.

Hay que tener cuidado especialmente con:

|safe

131. Nunca confiar en datos del cliente

Si el formulario envía:

```
owner_id=5
```

no deberíamos asumir que es correcto.

Para crear un objeto del usuario actual:

```
tournament.owner = (
    request.user
)
```

No:

```
tournament.owner_id = (
    request.POST["owner_id"]
)
```

PARTE XXXVII — IDEAS “WOW” PARA MOSTRAR EN CLASE

132. Mapa de jugadores conectados

Django
+

Django más allá del CRUD

geolocalización

+

Leaflet

Mostrar:

-  Madrid — Ana
 -  Toledo — Pedro
 -  Valencia — Marta
-

133. Recomendador de torneos

Combinar:

usuario

ubicación

intereses

historial

Machine Learning

Resultado:

Torneos recomendados para ti

134. Generador de contenido con IA

Botón:

🌟 Generar descripción con IA

Formulario:

Tema:

Hackathon Django

Nivel:

Principiante

OpenAI devuelve borrador.

El usuario lo revisa antes de publicar.

Django más allá del CRUD

135. Calendar automático

Botón:

Añadir a Google Calendar

Evento creado con:

nombre

fecha

hora

ubicación

descripción

136. Generar certificado PDF

Cuando termina torneo:

Resultado

↓

Django

↓

PDF

↓

Certificado de participación

137. Exportar ranking a Excel

Botón:

Exportar Excel

Generar:

ranking.xlsx

con:

Jugador

Partidas

Victorias

Django más allá del CRUD

Puntuación
Ranking

138. QR

Podemos instalar:

```
pip install qrcode
```

Generar un QR para:

entrada de torneo
perfil
check-in
enlace

Ejemplo:

```
import qrcode
```

```
img = qrcode.make(  
    "https://duckyarena.com/tournament/12/"  
)
```

```
img.save(  
    "tournament.png"  
)
```

139. Slugs

En lugar de:

```
/tournament/27/
```

podemos utilizar:

```
/tournament/ducky-python-cup/
```

Modelo:

```
slug = models.SlugField(  
    unique=True
```

Django más allá del CRUD

)

Esto resulta más legible y mejor para URLs públicas.

140. UUID

En lugar de IDs predecibles:

```
1
2
3
4
```

podemos utilizar UUID.

```
import uuid
```

```
id = models.UUIDField(
    primary_key=True,
    default=uuid.uuid4,
    editable=False
)
```

141. Soft delete

En vez de borrar realmente:

```
Tournament.objects.delete()
```

podemos añadir:

```
is_active = models.BooleanField(
    default=True
)
```

Y “eliminar”:

```
tournament.is_active = False
```

```
tournament.save()
```

Django más allá del CRUD

Esto sirve para explicar una práctica habitual en sistemas donde necesitamos conservar históricos.

142. Timestamps

Muy recomendable:

```
created_at = models.DateTimeField(  
    auto_now_add=True  
)
```

```
updated_at = models.DateTimeField(  
    auto_now=True  
)
```

Así sabemos:

cuándo se creó
cuándo se modificó

143. Abstract Model

Si todos nuestros modelos necesitan fechas:

```
class TimeStampedModel(  
    models.Model  
):
```

```
    created_at = (  
        models.DateTimeField(  
            auto_now_add=True  
        )  
    )
```

```
    updated_at = (  
        models.DateTimeField(  
            auto_now=True  
        )  
    )
```

```
    class Meta:
```

Django más allá del CRUD

```
abstract = True
```

Entonces:

```
class Tournament(
    TimeStampedModel
):
    ...
```

Evita repetir código.

144. Propiedades calculadas

```
@property
def available_places(self):

    return (
        self.max_players
        -
        self.players.count()
    )
```

Template:

```
{{ tournament.available_places }}
```

145. Métodos en modelos

No toda lógica pertenece a la view.

```
class Tournament(models.Model):

    ...

    def is_full(self):

        return (
            self.players.count()
            >=
            self.max_players
        )
```

Django más allá del CRUD

Uso:

```
if tournament.is_full():  
  
    messages.error(  
        request,  
        "Torneo completo."  
    )
```

PARTE XXXVIII — PROPUESTA DE DEMO COMPLETA

146. Caso práctico

Un usuario entra en DuckyArena.

1. Login

Django crea/reconoce su sesión.

2. Dashboard

Middleware mide tiempo.

3. Busca torneo

Utilizamos:

Q

Paginator

select_related

4. Activa geolocalización

JavaScript obtiene coordenadas.

5. Ve mapa

Leaflet muestra torneos próximos.

6. Se inscribe

Django comprueba permisos y plazas.

Django más allá del CRUD

7. Mensaje

Inscripción realizada correctamente.

8. Email

Recibe confirmación.

9. Calendar

Añade evento a Google Calendar.

10. IA

Puede pedir un resumen de reglas.

11. ML

El organizador ve predicción de asistencia.

12. Logging

El servidor registra acciones importantes.

Así los alumnos empiezan a percibir Django como el centro de un ecosistema, no como un simple generador de páginas HTML.

PARTE XXXIX — QUÉ GUARDARÍA PARA DRF

No explicaría profundamente todavía:

Serializers

APIView

GenericAPIView

ViewSets

Routers

DRF permissions

JWT

Token Authentication

API pagination

API filtering

OpenAPI

Swagger

Pero sí enseñaría hoy:

Django más allá del CRUD

```
return JsonResponse({  
    "message": "Hola"  
})
```

Y terminaría con la pregunta:

¿Y si toda nuestra aplicación
devolviese JSON en lugar de HTML?

La respuesta:

Django REST Framework

147. Librerías que merece la pena que conozcan

Aunque no haya tiempo para implementarlas todas:

Librería	Para qué sirve
<code>django-debug-toolbar</code>	Depuración y consultas SQL
<code>django-extensions</code>	Herramientas adicionales
<code>Pillow</code>	Imágenes
<code>requests</code>	APIs HTTP
<code>httpx</code>	HTTP sync/async

Django más allá del CRUD

<code>python-dotenv</code>	Variables de entorno
<code>django-allauth</code>	Login social
<code>django-crispy-forms</code>	Formularios
<code>Celery</code>	Tareas asíncronas
<code>Redis</code>	Caché y colas
<code>Channels</code>	WebSockets
<code>pandas</code>	Datos
<code>openpyxl</code>	Excel
<code>scikit-learn</code>	Machine Learning
<code>qrcode</code>	QR
<code>openai</code>	Modelos OpenAI
<code>google-api-python-client</code>	APIs Google

Django más allá del CRUD

Faker	Datos ficticios
factory_boy	Datos de tests
WhiteNoise	Static files
gunicorn	Servidor WSGI Linux
psycopg	PostgreSQL

148. Mis “tips” que deberían memorizar

Tip 1

No guardar secretos en código.

.env

Tip 2

No confiar en botones ocultos como sistema de permisos.

Proteger siempre la view.

Tip 3

No utilizar:

request.user.role

si `role` pertenece a `ProfileUser`.

Usar:

request.user.profile.role

Django más allá del CRUD

Tip 4

No utilizar:

```
password="123456"
```

con `User.objects.create()`.

Usar:

```
create_user()
```

o:

```
set_password()
```

Tip 5

Para objetos:

```
get_object_or_404()
```

Tip 6

Para comprobar existencia:

```
.exists()
```

Tip 7

Para ForeignKey/OneToOne que vamos a consultar:

```
select_related()
```

Tip 8

Para ManyToMany:

```
prefetch_related()
```

Tip 9

No mostrar miles de objetos.

Usar:

Paginator

Django más allá del CRUD

Tip 10

No usar `print()` como solución permanente.

Utilizar:

logging

Tip 11

No meter toda la lógica en views.

Considerar:

services.py
decorators.py
signals.py
utils.py

Tip 12

No duplicar URLs manualmente.

Utilizar:

{% url %}

y:

reverse()

Tip 13

No consultar una API externa desde el navegador si eso implica exponer una API key privada.

Hacer:

Browser

↓

Django

↓

API externa

Tip 14

No ejecutar operaciones muy lentas en una request si pueden convertirse en tareas asíncronas.

Django más allá del CRUD

Tip 15

Antes de optimizar:

medir

Herramientas como `django-debug-toolbar` ayudan a saber dónde está realmente el problema.

149. Posible estructura de una sesión larga

Primera parte

Configuración profesional:

.env
.gitignore
settings
logging
email

Segunda parte

Usuarios:

decoradores
roles
groups
permissions
messages
sessions

Tercera parte

Django interno:

middleware
signals
context processors

Django más allá del CRUD

template filters

Cuarta parte

ORM:

Q

F

annotate

aggregate

select_related

prefetch_related

Paginator

Quinta parte

Demostraciones llamativas:

mapa

geolocalización

QR

Excel

PDF

Sexta parte

Integraciones:

Google Calendar

OpenAI

Machine Learning

APIs externas

Séptima parte

Arquitectura moderna:

cache

Redis

Celery

WebSockets

PWA

Django más allá del CRUD

Final

JsonResponse

↓

"¿Y cómo hacemos una API completa?"

↓

DRF

150. Mensaje final para los alumnos

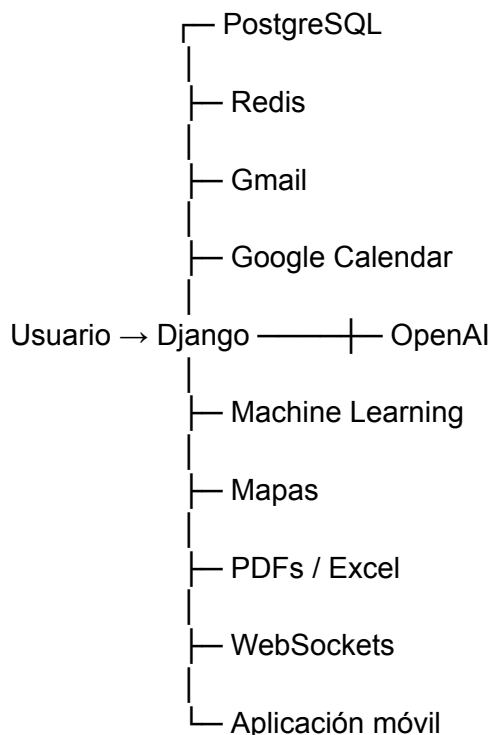
Django no es solamente:

Model

View

Template

Django puede convertirse en el núcleo de un sistema mucho mayor:

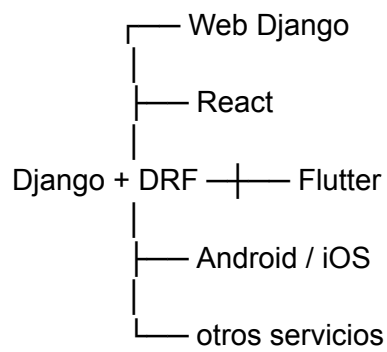


El concepto importante es entender que Django puede funcionar como el **centro coordinador de una aplicación**.

Gestiona usuarios, permisos, modelos y reglas de negocio y, cuando es necesario, se comunica con otros servicios.

La semana siguiente añadiremos una pieza fundamental:

Django más allá del CRUD



Y el elemento que permitirá esa comunicación será una API.